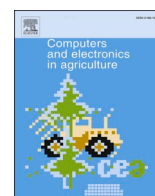




Contents lists available at ScienceDirect

Computers and Electronics in Agriculture

journal homepage: www.elsevier.com/locate/compag



Real-time strawberry detection using deep neural networks on embedded system (rtsd-net): An edge AI application

Yanchao Zhang^{a,b,*}, Jiya Yu^a, Yang Chen^{b,c}, Wen Yang^a, Wenbo Zhang^a, Yong He^{b,*}

^a School of Machinery Engineering and Automation, Zhejiang Sci-Tech University, Hangzhou, Zhejiang Province 310018, China

^b College of Biosystem Engineering and Food Science, Zhejiang University, Hangzhou, Zhejiang Province 310058, China

^c Zhejiang Lab, Hangzhou, Zhejiang Province 310018, China

ARTICLE INFO

Keywords:

YOLO
Object detection
Deep learning
Edge computing

ABSTRACT

Computer vision is a key technique to make agricultural machinery smart. Deep neural network has achieved great success in computer vision. How to use it at a small size, low cost, low power consumption device with high accuracy and speed on strawberry harvesting machinery has drawn much research attention. Since the infield situation has reduced number of objects and that they are easier to be distinguished from the background compared to other computer vision datasets, the huge neural network structure can be simplified in order to speed up the detection inference without penalizing the detection accuracy. In this research, a new deep neural network called RTSD-Net is proposed based on state-of-art light-weighted YOLOv4-tiny with reduced layers and modified structure for real-time strawberry detection under infield condition. The original CSPNet was replaced by 2 types of CSPNet designed with reduced parameters and a simplified structure and 4 new network structures are designed by combining these 2 types. The performances of the 4 networks were evaluated. It was observed that the number of parameters of these 4 networks and the detection speed of the model is negatively correlated. Simplified structure and reduced parameters can contribute to faster operational speed. The last one was selected and named as RTSD-Net. Comparing with YOLOv4 tiny, the accuracy of RTSD-Net is only reduced by 0.62% but the speed is increased by 25FPS, which is 25.93% higher than that of YOLOv4-tiny. Embedded system Jetson Nano was selected as the evaluation platform to evaluate the RTSD-Net's performance for edge computing. The original Open Neural Network Exchange (ONNX) model was loaded on Jetson Nano and the speed of RTSD-Net was 13.1FPS, which is 19.0% higher than that of YOLOv4-tiny. After speeded up by TensorRT method, the transformed model reached 25.20fps, which is twice as fast as the ONNX model, and 15% faster than the YOLOv4-tiny model. After speeding up, the efficiency of RTSD-Net is enough for computer vision based strawberry detection and harvesting. In summary, the proposed RTSD-Net has good potential in smart strawberry harvesting machinery and the idea of redesigning neural structure and reducing parameters to speed up the detection rate of deep neural network is expected to have good application in edge computing.

1. Introduction

Machine learning is a hot topic in the field of artificial intelligence and it is the key technique for smart agricultural machinery. Deep

learning is the forefront of the development of machine learning which has achieved excellent results in many ways like natural language processing and computer vision. Deep convolutional neural networks (DCNNs) have made significant progress in computer vision, which have

Abbreviations: AI, Artificial Intelligence; AP, Average Precision; CBL, Conv Batch Normalization Leakrelu; CIoU, Complete Intersection over Union; CNNs, Convolutional Neural Networks; CSI, CMOS Sensor Interface; CSPNet, Cross Stage Partial Network; CUDA, Compute Unified Device Architecture; DC, Direct Current; DIoU, Distance Intersection over Union; FLOPs, Floating Point Operations; FP, False Positives; FP16, Half-precision floating-point; FPN, Feature Pyramid Network; FPS, Frames Per Second; FN, False Negatives; GPU, Graphics Processing Unit; HIDM, High-Definition Multimedia Interface; IoU, Intersection over Union; mAP, Mean Average Precision; ONNX, Open Neural Network; PAN, Path aggregation network; PC, Personal Computer; SPP, Space Pyramid Pool; TP, True Positives; UAV, Unmanned Aerial Vehicle; USB, Universal Serial Bus.

* Corresponding authors at: School of Machinery Engineering and Automation, Zhejiang Sci-Tech University, Hangzhou, Zhejiang Province 310018, China (Y. Zhang).

E-mail addresses: yczhang@zstu.edu.cn (Y. Zhang), yhe@zju.edu.cn (Y. He).

<https://doi.org/10.1016/j.compag.2021.106586>

Received 15 July 2021; Received in revised form 19 November 2021; Accepted 24 November 2021

Available online 7 December 2021

0168-1699/© 2021 Elsevier B.V. All rights reserved.

greatly improved the performance of image classification, segmentation and detection. In terms of object detection, the accuracy of deep learning algorithms can be higher than visual recognition with incomparable efficiency. In the latest results, Deep learning algorithms have achieved 97% accuracy in the identification and positioning of rhizome features (Pound et al., 2018).

The current development of deep learning has two directions. One is to develop towards high precision, and the other is to develop towards high efficiency. Alexey AB (Bochkovskiy et al., 2020) proposed YOLOv4 with more prominent accuracy on the basis of YOLOv3, which improved data processing, network structure and loss function to achieve better detection accuracy. In addition to the accuracy improvement, the improvement in speed is also of great significance. Andrew (Howard et al., 2017) proposed the deep separable convolution network, and constructed the lightweight network MobileNet-v2 to greatly reduce the network parameters. In terms of the results, the accuracy of the network has declined to a certain extent while the detection speed has been greatly improved, which made it meet the requirements of specific scenarios. Based on YOLOv4 network, Alexey A.B proposed YOLOv4-tiny lightweight network, whose speed can reach 371 FPS when running on PC with GTX 1080 Ti. Besides, the model can also be deployed on embedded mobile devices, such as Jetson Nano or mobile smart phone. Lightweight network greatly promotes the deployment process of the model on edge computing devices.

Computer vision has experienced great achievement recently in agriculture automation and deep learning. For example, Chen (M. (Chen et al., 2021) obtained a detailed 3D orchard map combining eye-hand stereo vision and SLAM system. The candidate fruits are obtained by partial matching of contour information and probabilistic Hough transform, and then trained by SVM classifier. The experiment has good detection effect in most types of fruits (Lin et al., 2019). However, in smart agricultural machinery, many tasks need to be executed quickly on mobile devices like embedded systems, which makes researchers pay attention to the development of light weight deep neural network in edge computing. Deep learning algorithm has been used to identify crops, weeds and diseases in agriculture. (Y. (Chen et al., 2019) used UAV and RGB cameras to shoot strawberry fields, and used Faster-RCNN algorithm to detect the targets after segmentation. The number of strawberries and flowers was calculated to estimate the yield and accuracy rate has reached 84.1%. This research helps farmers to observe the growth cycle of strawberry fields in a more efficient way. (Gao et al., 2020) conducted research based on YOLOv3-tiny network to improve the speed of object detection and manage weeds in sugar beet field, which meets the requirements of speed and accuracy. Liu (Liu and Wang, 2020) proposes an early recognition method of tomato leaf spot based on MobileNetv2-YOLOv3 model to achieve a good balance between the accuracy and real-time detection of tomato gray leaf spot. Aguiar used deep learning algorithms such as MobileNets, Inception and lite YOLO to detect the trunk in real-time detection as an experimental performance benchmark between two devices is performed: NVIDIA's Jetson Nano and Google's USB Accelerator (Aguiar et al., 2020). Mazzia carried out the benchmark tests on two experimental devices, the Jetson Nano of NVIDIA and the USB Accelerator of Google for the real-time detection of apples in orchards and the evaluation of apple production (Mazzia et al., 2020).

Strawberries are of good taste and rich nutritional value, like multiple vitamins and amino acids (Proteggente et al., 2002). It's well loved by the people all over the world. The worldwide consumption is expected to reach 11.5 million tons. Among all producing countries China has the largest production and consumption. However, this industry is very labor-intensive, especially in harvesting, which makes the price of strawberries so high. Strawberry fruits should be harvested as soon as possible after maturity, but the problem is, the workload is so heavy that it is impossible to finish the harvest in such a short time. So, it is very meaningful to use automatic equipment to harvest strawberry. From this aspect, reducing labor force can really bring significant economic

benefits

One key technique for smart harvesting is real-time object detection. How to improve the recognition speed of strawberries while ensuring the recognition accuracy of strawberries has become the focus of this work. Real-time detection algorithm is of great research significance in agriculture. In the single agricultural scene, the variety of object is far lower than some public datasets, such as COCO datasets. In the case of simple scene and fewer types of objects, how does reducing the complexity of network structure affect the accuracy and speed of object detection? In intelligent agricultural machinery, the improvement of detection speed is valuable. Although some researchers work on edge computing methods to deploy deep neural network on mobile platform, few of them works on the CNN structure to improve performance. The deep neural network consists of millions of parameters and sophisticated neural structures. Which parameter can be reduced and how is the effect on efficiency? This is a big issue in building DNN since some change may lead to bad effect.

In this research, we aim to find out a way to improve the performance of lightweight deep neural network and deploy it on embedded platform. And we aim to find out what can be done to the internal structure of YOLOv4-tiny so the speed can increase without compromising accuracy.

2. Materials and methods

2.1. System setup

As the Fig. 1 shows, UAV was used for fast collection of data over the strawberry field. A total of 3 categories, strawberry, immature strawberry and flower were marked. Before the data set was put into the network, the data set needs preprocessing. On the network structure, we compared the detection results of 8 kinds of networks. 4 of them are YOLOv3, YOLOv3-tiny, YOLOv4, and YOLOv4-tiny, and the rest are the improved network structures. The corresponding speed was obtained by inference on PC and Jetson Nano devices. At the same time, the PTH format model was transformed into the TRT serialization format model, and then loaded on Jetson Nano, as shown in Fig. 1. The strawberry dataset and trained model can be downloaded at the following link:

<https://drive.google.com/drive/folders/1l8V4yhOJ5ZYkwx6SIArU2yr85KcJSXBV?usp=sharing>.

2.2. Data collection and preprocessing

A strawberry field (located at 29.404265° N, 82.141893° W) was prepared at the Plant Science Research and Education Unit (PSREU) at the University of Florida in Citra, Florida, USA during the 2017–2018 growing season. The drone that collects the strawberry data set is DJI Phantom 4 Pro (DJI Technology Co. Ltd., Shenzhen, China). The strawberry experiment field were 67 m long and six meters wide with five rows of strawberry plants, each being 67 m long and 0.5 m wide. The drone works fully automatically, as long as the target area and flight parameters are pre-set at the ground control station. 3 m height for image acquisition was chosen to cover 3 rows of strawberries since the strawberry fruit and flowers are relatively small. In bounding box annotation for object localization and detection, the rectangular borders with different colors were used to mark 3 different objects, mature strawberry, immature strawberry and flower, respectively. All labels were created manually using the LabelImg software developed by the Computer Science and Artificial Intelligence Laboratory (MIT, MA, USA). And the positive image is segmented into a small rectangular image of 480 × 380 pixels to improve the inference speed of yolo model.

2.3. Hardware selection

Jetson Nano is a small, powerful computer for embedded AI applications like edge computing and robotics, which was launched by

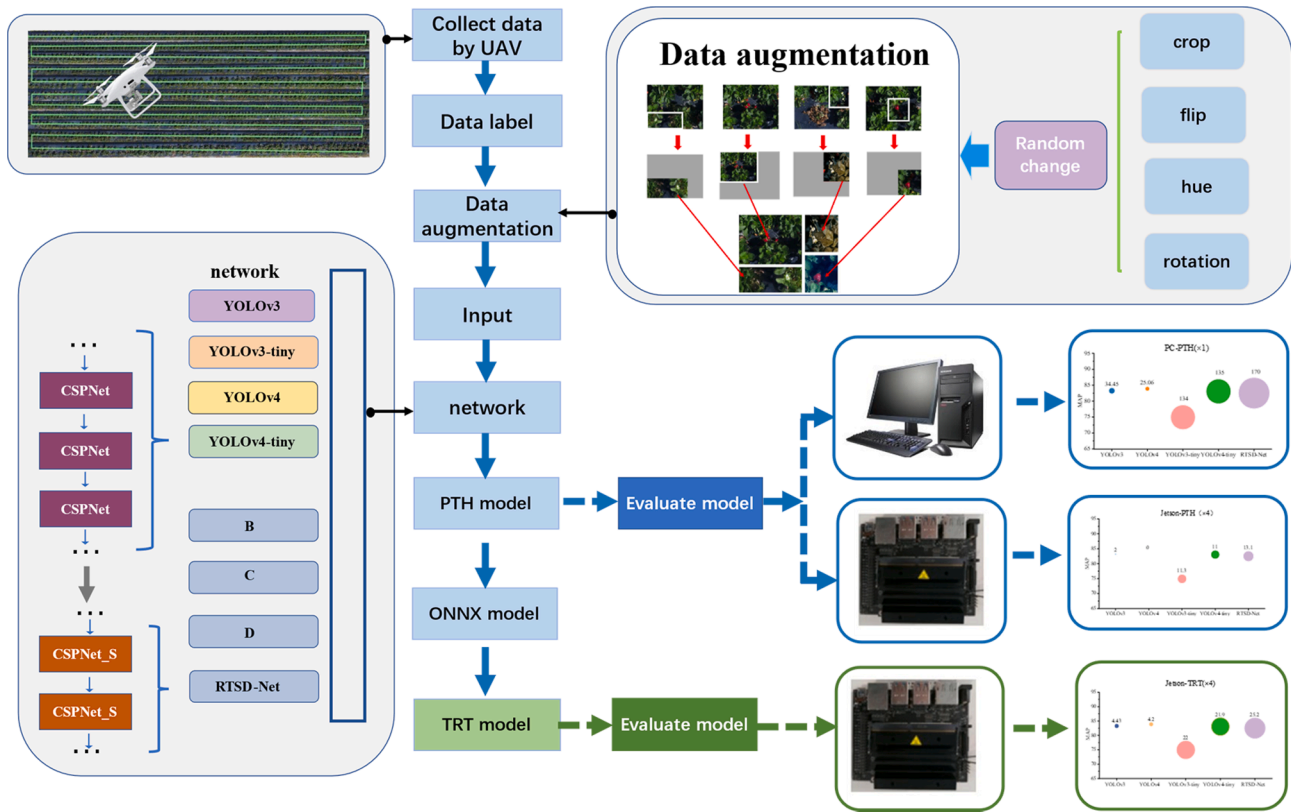


Fig. 1. Flowchart of strawberry detection network training deployment.

NVIDIA (NVIDIA Corporation, USA) in March 2019. With a Quad-core ARM Cortex-A57 CPU and Maxwell architecture 128-core GPU, it can bring enough computing power for AI applications like vehicle navigation (Cass, 2020); (Sati et al., 2020); (Vijitkunsawat and Chantngarm, 2020). And thanks to the abundant interfaces provided by NVIDIA for efficient implement of deep neural networks, Jetson Nano has become an ideal embedded platform for edge computing. It supports NVIDIA's CUDA, cuDNN and TensorRT software libraries, and a series of popular AI frameworks and algorithms, such as PyTorch, Caffe, MXNet, etc. The typical power supply regulated by NVIDIA is 4A@5V, which meets the power supply requirements of the trained model. Both CSI and USB cameras are available on Jetson Nano to support computer vision tasks. Anyway, Jetson Nano is an excellent embedded developer kit that meets the hardware conditions for model deployment.

2.4. Neural network selection

Object detection algorithms are evolving rapidly in recent years, whose accuracy has improved significantly. YOLO is one of the classic one-stage object detection algorithms that has been adopted by many researchers and gone through 4 generations. The latest version is YOLOv4. The basic concept of YOLO is to convert the object detection to regression to reach higher efficiency and accuracy. Its key method is to divide the input picture into $S \times S$ grids. Each grid is responsible for detecting the targets whose center point falls in the grid. However, YOLO network is too big and structure is too complicated to be implemented on embedded or mobile devices. In order to deal with this problem, researchers proposed lightweight YOLO series, including YOLOv2-tiny, YOLOv3-tiny, and YOLOv4-tiny.

2.4.1. Classic YOLO network structure

The backbone network of YOLOv3 (Redmon and Farhadi, 2018) is darknet53 which uses 53 convolution layers, and with the addition of the residual blocks, it can solve the problems of vanishing gradient and

network degradation when the number of network layers is too deep. Also, there are three prediction branches existing in YOLOv3, and each of them has 3 priori boxes, which are obtained by using a clustering algorithm.

The backbone network of YOLOv4 adds Cross Stage Partial Network (C.-Y. (Wang et al., 2020) on the basis of YOLOv3. A large residual edge is added outside the stacked residual block of darknet53 to form the Cross Stage Partial Network. Cross Stage Partial Network consists of two parts, the main part continues to stack the original residual blocks, and the other part is directly connected to the final after a small amount of processing. This operation can fuse information between different layers, avoid that the different layers learn the same gradient information. It reduces the amount of calculation, and obtain higher accuracy. YOLOv4 network uses path aggregation network (PAN) (W. (Wang et al., 2019) and space pyramid pool (SPP) (He et al., 2015).

The backbone network of YOLOv3-tiny has only seven convolution layers and six max pooling layers. Compared with the 53 convolution layers of YOLOv3, the number of network layers is significantly reduced. In the output branch, it only has two scales predictions, and the output branch of YOLOv4-tiny also retains two scales predictions, one less than the output branch of YOLOv3 algorithm. Although this processing loses part of the accuracy, it has greatly improved the speed and obtained great advantages in edge computing.

YOLOv4-tiny is designed based on YOLOv4, whose backbone network is CSPDarknet53-tiny, which only contains three convolution layers and three cross stage partial networks. In the feature extraction network, the space pyramid pool (SPP) and path aggregation network (PAN) of YOLOv4 are deleted, and the feature pyramid network (FPN) is used. Comparing with the residual block in YOLOv3, the CSPNet learning ability is greatly improved. The excellent performance of YOLOv4-tiny is related to the CSPNet in the backbone network.

2.4.2. Improvement of CSPNet

In the strawberry data set, there are only three types of objects,

mature strawberry, immature strawberry, and strawberry flower. In contrast with the COCO data set, the number of categories is greatly reduced, which is supposed to reduce the difficulty of network learning. Embedded devices are usually very small, and the corresponding performance is greatly reduced. Therefore, the simpler the network structure it is, the more conducive to the deployment of it on embedded devices. To simplify the network structure, we can consider from two aspects. On the one hand, we can reduce the number of the CSPNet in the backbone network. On the other hand, it reduces the complexity of the CSPNet. Since there are only 3 CSPNets in YOLOv4-Tiny, reducing the CSPNet's number must be done very discreetly. We designed two cross stage partial networks named CSPNet_F and CSPNet_S to replace the CSPNet. The structures of these 3 types of CSPNet are shown in Fig. 2. In the CSPNet_F network, the feature map passes through a 3×3 convolutional layer, and then enters a dual-path network. The first path does not undergo any processing, and the second path divides the number of channels into half of the original. After a residual block and convolution layer, it adds the feature map of the first path to complete the expansion of the number of channels. Finally, it compresses the size of the feature map into half of the original through the maximum pooling layer. In CSPNet_S module, a convolution layer is deleted on the

basis of CSPNet_F.

In order to compare the complexity of the three networks, we use the floating-point operations (FLOPs) to represent the parameters of the module. The calculation is shown in Eq. (1).

$$FLOPs = \sum_{i=1}^N L_i^2 \times K_i^2 \times C_{i-1} \times C_i \quad (1)$$

Where N is the number of convolutional layers, L_i is the output feature layer size of the i th layer convolution. K_i is the number of convolution kernels. C_i and C_{i-1} are the number of output channels and input channels respectively.

We take the input feature layer size of 104×104 and 64 channels as an example for calculation. As shown in Table 1-3.

It can be seen from the above tables that the FLOPs amount of CSPNet_F is reduced by 15.49% comparing with CSPNet, and the number of parameters of CSPNet_S is reduced by 22.37% compared with CSPNet.

2.4.3. Design and comparison of backbone networks

Backbone network is a process of image size compression and

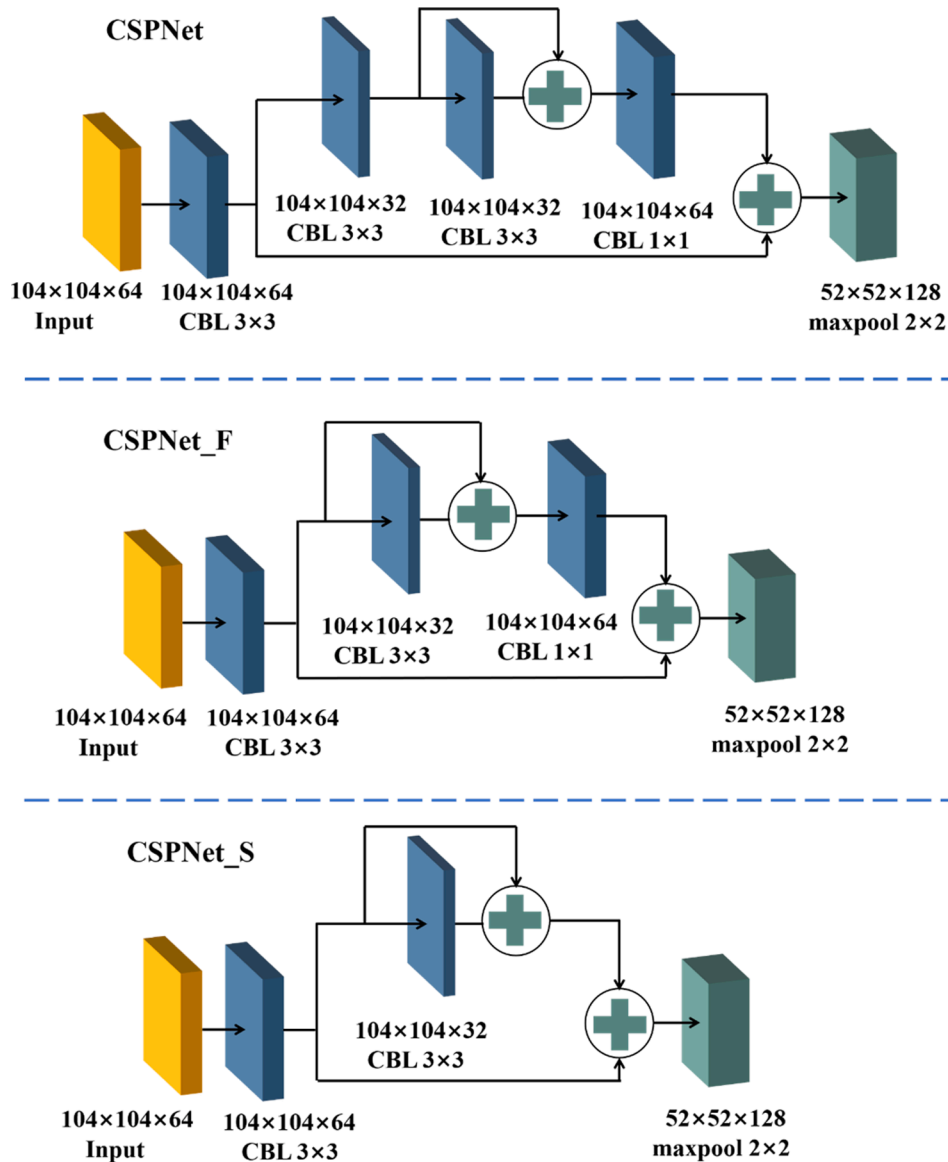


Fig. 2. Network structure diagram of CSPNet, CSPNet_F and CSPNet_S.

Table 1

The FLOPs of the CSPNet module.

Type/Stride	Filter Shape	Size	FLOPs (10 ⁸)
Conv/S = 1	3*3*64	104*104	3.987
Conv/S = 1	3*3*32	104*104	0.997
Conv/S = 1	3*3*32	104*104	0.997
Conv/S = 1	1*1*64	104*104	0.443
Maxpool	2*2*128	52*52	0.014
Total			6.438

Table 2

The FLOPs of the CSPNet_F module.

Type/Stride	Filter Shape	Size	FLOPs (10 ⁸)
Conv/S = 1	3*3*64	104*104	3.987
Conv/S = 1	3*3*32	104*104	0.997
Conv/S = 1	1*1*64	104*104	0.443
Maxpool	2*2*128	52*52	0.014
Total			5.441

Table 3

The FLOPs of the CSPNet_S module.

Type/Stride	Filter Shape	Size	FLOPs (10 ⁸)
Conv/S = 1	3*3*64	104*104	3.987
Conv/S = 1	3*3*32	104*104	0.997
Maxpool	2*2*128	52*52	0.014
Total			4.998

channel number expansion. Based on the proposed 2 new cross-stage network structures CSPNet_F and CSPNet_S, we designed 4 new backbone network structures to simplify the network by gradually reducing the number of convolutional layers in the network, and named B, C, D and E respectively. Fig. 3 shows these 5 network structures. B replace the third CSPNet of YOLOv4-tiny with a convolution layer. C removed the

last convolution layer of B. D replaced the 2 CSPNets of C with CSPNet_F. E replaced the 2 CSPNets of C with CSPNet_S. As we can see, the structure of network E is the simplest, and we name it RTSD-Net. In the backbone network of RTSD-Net, there are only 3 convolutional layers and 2 cross-stage network structure CSPNet_S. Under the condition that the size of the obtained feature layer is constant, the number of convolution layers is reduced to the maximum extent possible, so as to minimize the calculation amount of the network. RTSD-Net has only seven convolution layers and two pooling layers in the backbone network, which is even simpler than the number of network layers of YOLOv3-tiny.

The FLOPs of 5 networks are shown in the Table 4. Based on the YOLOv4-tiny network, a series of modifications are carried out, and 4 new networks are proposed. The number of network parameters is gradually reduced, and the number of parameters in network E is reduced by 44.05% from YOLOv4-tiny network.

2.5. Loss function

IoU is the degree of overlap between the predicted result and the bounding box in the original image. It is the most common indicator in object detection. Its function is not only to determine the positive sample and the negative sample, but also to evaluate the distance between the predicted box and the target box. The shortcoming of IoU is that if the two bounding boxes do not intersect, IoU = 0, which can't reflect the

Table 4

FLOPs of Five different networks improved based on YOLOv4-tiny.

Network	CBL	CSPNet	CSPNet_F	CSPNet_S	FLOPs (10 ⁸)
A	3	3	\	\	25.65
B	4	2	\	\	21.22
C	3	2	\	\	17.23
D	3	\	2	\	15.24
E	3	\	\	2	14.35

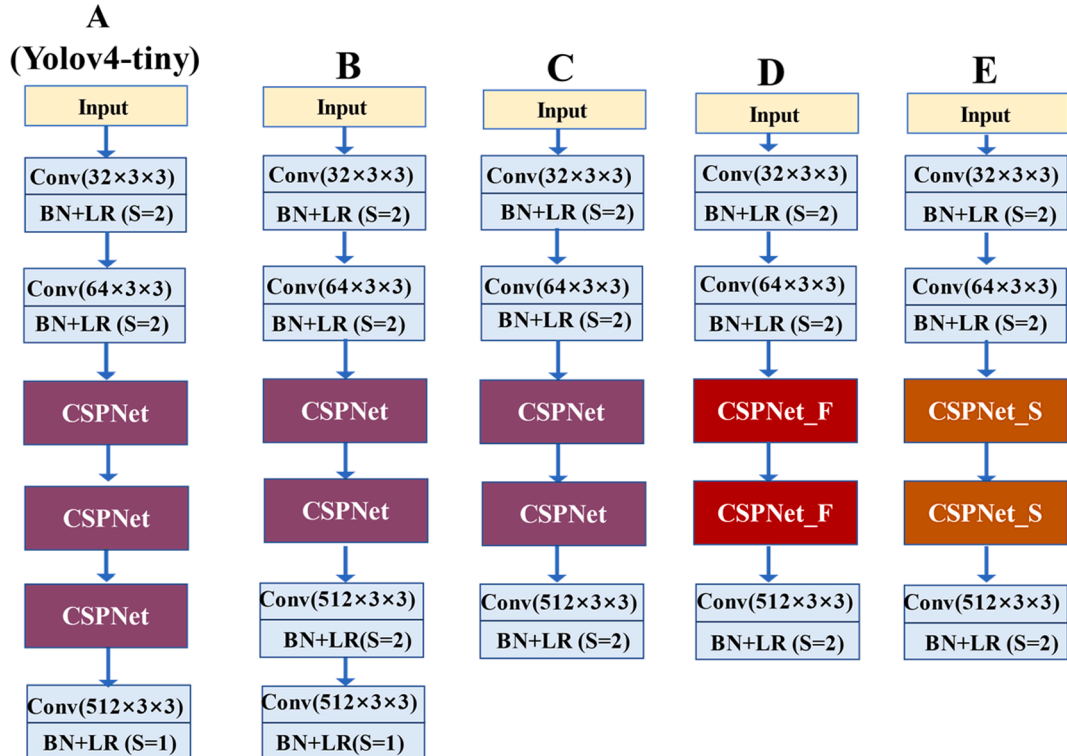


Fig. 3. Backbone network structure. A is the backbone network YOLOv4 -tiny, B, C, D and E are the improved network structures based on A.

distance between the two bounding boxes. Learning and training can't be performed without gradient backhaul. On the basis of IoU, researchers have proposed GIoU, DIoU and CIoU (Zheng et al., 2020). GIoU solves the problem of IoU when the target box does not coincide with the predicted box. DIoU considers central point distance and overlap area of the 2 bounding boxes. CIoU is more comprehensive, and the aspect ratio is additionally considered on the basis of DIoU. CIoU is defined as Eq. (2).

$$CIoU = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha\nu \quad (2)$$

Where b and b^{gt} denote the central point of predicted box and target box, $\rho(\cdot)$ show the Euclidean distance between the two boxes. c is the diagonal length of the smallest closed box which cover the 2 bounding boxes. The expression of ν is defined as Eq. (3), α is shown in Eq. (4).

$$\nu = \frac{4}{\pi^2} (\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h})^2 \quad (3)$$

$$\alpha = \frac{\nu}{(1 - IoU) + \nu} \quad (4)$$

Where w^{gt} and h^{gt} denote the width and height of the target box, w and h is the width and height of the predict box.

The function of this penalty item is to control the width and height of the predicted box to be as close as possible to the target box. If the width and height of the target box and the predicted box are similar, then ν is 0 and the penalty item will not work. The CIoU loss function considers the geometric factors between the prediction box and the target box more comprehensively. The CIoU loss function is used for training in this research.

The loss function of YOLOv4-tiny consists of 3 parts. As shown in Eq. (5)

$$loss = loss_1 + loss_2 + loss_3 \quad (5)$$

Where $loss_1$, $loss_2$ are $loss_3$ confidence loss function, classification loss function and bounding box regression loss function, respectively. The confidence loss uses a cross-entropy loss function. Confidence loss has two situations, namely, there are targets and no targets in the bounding box. as shown in Eq. (6).

$$loss_1 = - \sum_{i=0}^{S^2} \sum_{j=0}^B W_{ij}^{obj} [D_i^j \log(C_i^j) + (1 - D_i^j) \log(1 - C_i^j)] - \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B (1 - W_{ij}^{obj}) [D_i^j \log(C_i^j) + (1 - D_i^j) \log(1 - C_i^j)] \quad (6)$$

Where S^2 is the number of grids in the input image, B is the number of bounding boxes in the grid, W_{ij}^{obj} is only the function of objects. If the j th bounding box of the i grid is responsible for detecting the current object, then, $W_{ij}^{obj} = 1$ otherwise $W_{ij}^{obj} = 0$. C_i^j and D_i^j are the confidence values of prediction box and target box respectively. λ_{noobj} is the weight parameter.

Classification loss functions are show in Eq. (7):

$$loss_2 = - \sum_{i=0}^{S^2} \sum_{j=0}^B W_{ij}^{obj} \sum_{c=1}^C [Q_i^j(c) \log(P_i^j(c)) - (1 - Q_i^j(c)) \log(1 - P_i^j(c))] \quad (7)$$

Where $P_i^j(c)$ and $Q_i^j(c)$ are the prediction probability and real probability of the object belonging to the correct category in the j bounding box of the i grid.

In order to more accurately represent the spatial relationship between the predicted box and the target box, when calculating the boundary regression loss function, the CIoU loss function expression is used to replace the IoU.

$Loss_3$ equals CIoU. The calculation is as the Eq. (2) shows.

2.6. Training procedure

In this research, the experiment used the strawberry data set for network training and testing. The dataset has 3 categories, mature strawberry, immature strawberry and flower. The training environment of this work was as follows: the operational system is 64-bit Windows10, the processor is AMD Ryzen 5 3600 6-Core Processor, and the GPU is RTX3070. GPU can accelerate the training of the model. The versions of CUDA and CUDNN provided by NVIDIA are 11.1 and 8.0.4. The experiment used the PyTorch 3.7 framework for model building and training. The PyTorch framework is user-friendly and easy to use to build a deep learning network, and has strong flexibility. During training, first freeze the backbone network for training, the number of training epochs was 200, the batch size was 32, and the learning rate was $1e-3$, and then unfreeze the backbone network to continue training, the number of training epoch was 200, the batch was 16, and the learning rate was $1e-4$.

Techniques including mosaic augmentation method, cosine annealing learning rate, and label smoothing are used in training (Müller et al., 2019). Mosaic data augmentation stitch the 4 pictures together to get a new picture. Then these new pictures are passed into the neural network for learning, in this way, in a batch process, the network can learn more picture feature information. In network training, data augmentation is an important method. It can speed up the network training process by enriching the information of the dataset. In single sample data augmentation methods mainly include color transformation and geometric transformation. In addition to the single sample data augmentation method, there are also multiple sample data augmentation, and new samples are generated through multiple samples, such as mixup (Zhang et al., 2017), Cutmix (Yun et al., 2019), cutout (DeVries and Taylor, 2017) and mosaic.

The CutMix method uses two images. First, the basic data of these two images are enhanced. For example, gamut transformation, geometric transformation. Then stitch the two pictures together. The principle of Mosaic data augmentation is consistent with the principle of CutMix method, only two images are replaced by four images. Because four pictures are used, the cropping area cannot be simply cropped randomly, but determined by using the upper left corner of the stitching position of the selected picture. The Mosaic method is shown in Fig. 4.

The cosine annealing learning rate is a method of adjusting the learning rate during the training process. Model training is the process of finding the minimum value of the loss function. But during the training process, the network is easily deceived by the local optimum point. How to escape from the locally optimum is an important issue. The cosine annealing learning rate is different from the traditional learning rate. As the number of iterations increases, the learning rate first drops sharply, and then rises sharply. Then it repeats continuously. Such violent fluctuations can make the network escape from local optimum point and find new best points.

Label smoothing is a regularization tool, which changes the target variable so that the prediction result of the model is no longer just a certain value. Let's take the cross-entropy loss function of confidence as an example. The target variable is usually a one-hot vector. The result is 1 when there is a target in the bounding box, otherwise the result is 0. the cross-entropy loss function and probability distribution are as Formula 8–9.

$$loss = -D_i \times \log C_i - (1 - D_i) \times \log(1 - C_i) \quad (8)$$

$$Z_i = \begin{cases} \infty, (D_i = C_i) \\ 0, (D_i \neq C_i) \end{cases} \quad (9)$$

Where C_i^j and D_i^j are the confidence values of prediction box and target box respectively.



Fig. 4. Data augmentation method, The white box represents the crop area.

When there is an object in the bounding box, the output probability value is infinite. The neural network will encourage itself to learn in the direction where the difference between the correct label and the wrong label is the largest, so that the confidence is infinitely close to 1. In our data set, the amount of data is large enough to show the characteristics of all samples to avoid over-fitting.

Label smoothing uses the weighted average of the hard target and the uniform distribution on the label as the soft target. The minimum value of the target vector is changed to α .

The model determines whether there is a target in the boundary box, and its result is no longer only 1 or 0, but $1-\alpha$ or α . T is any real number. The loss function and probability distribution obtained at this time are as Formula 10–11.

$$loss_i = \begin{cases} (1 - \alpha) \times loss, (D_i = C_i) \\ \alpha \times loss, (D_i \neq C_i) \end{cases} \quad (10)$$

$$Z_i = \begin{cases} \log \frac{(1 - \alpha)}{\alpha + T}, (D_i = C_i) \\ T, (D_i \neq C_i) \end{cases} \quad (11)$$

In this way, the network has both the correct label and the wrong label when learning. It reduces the weight of the true label category in the calculation of the loss function, thereby achieving the suppression of overfitting.

2.7. Model deployment

We used the PyTorch framework to train the network and a model in PTH format was generated. The model would be deployed on Jetson Nano to make real-time predictions. Jetson Nano supports TensorRT to accelerate the model. TensorRT is an accelerator library, which can improve the processing speed of neural networks by optimizing the algorithm architecture. TensorRT doesn't train the model itself, but only works as a complementary way for training frameworks. It uses GPU to quickly infer the trained network. Open Neural Network Exchange (ONNX) is an intermediate framework to bridge PyTorch model and

TensorRT model.

The optimization of TensorRT (Vanholder, 2016) has two main aspects. The first aspect is the fusion of layers and tensors, which integrates some operations. There are convolution layer, bias layer and reloading layer in the model before optimization. These three layers need to call API three times. To improve operating efficiency, the first step is the vertical integration of the network structure. We merged the conv, Batch normalization, and LeakRelu 3 layers together into 1 layer called CBL. The second step deletes the connection layer, which reduces the calculation of the connection layer. In addition to the above two operations, we also horizontally combine the network. Horizontal combination refers to the fusion of layers that are input into the same tensor and perform the same operation, as shown in Fig. 5.

The second aspect is to reduce the precision level of the network parameters. The precision of the network trained by the PyTorch framework is 32-bit floating point (FP32). In model training, there are two processes of forward propagation and back propagation, which require high precision network parameters. In model inference, there is only the process of forward propagation, which doesn't require high parameter precision and the precision of the parameters can be appropriately reduced. Jetson Nano supports FP16 model inference. There are two main methods to reduce network parameters precision from FP32 to FP16: symmetric mapping and asymmetric mapping. Both of them are shown in Fig. 6. The first step of symmetric mapping is to find the activated parameter. In the second step, the maximum value of the absolute value of the parameter is used as the threshold, and the range centered on the zero point is delineated. The third step maps this range to the FP16 area proportionally. Symmetric mapping is mainly used when the data is evenly distributed. However, the parameter distribution is not always uniform. If symmetric mapping is adopted in the meantime, the dynamic range of FP16 will be smaller, thus the loss of model accuracy will be great. Asymmetric mapping is proposed to solve this problem. It selects an asymmetric area among the activated parameters. z is the center of the area, also known as the adjusted zero point. The parameters in this area are mapped proportionally to the range of FP16. The parameter saturation outside the region is mapped to

CSPNet in yolov4-tiny

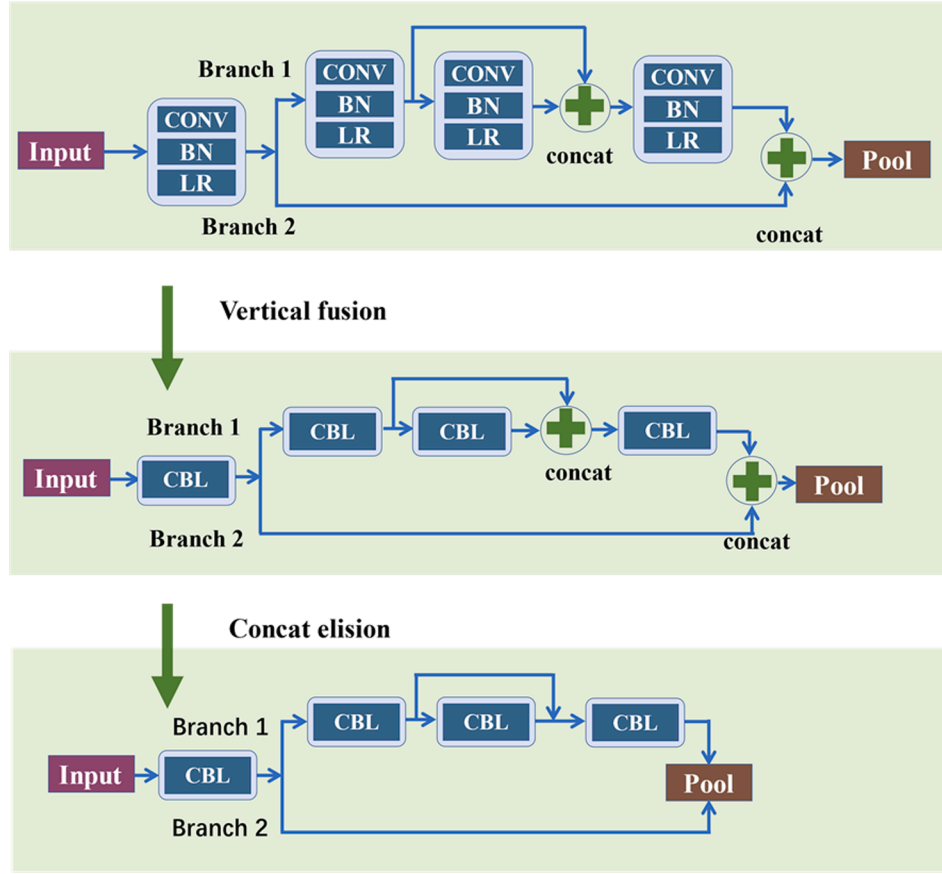


Fig. 5. The transformation of network structure is in TensorRT format.

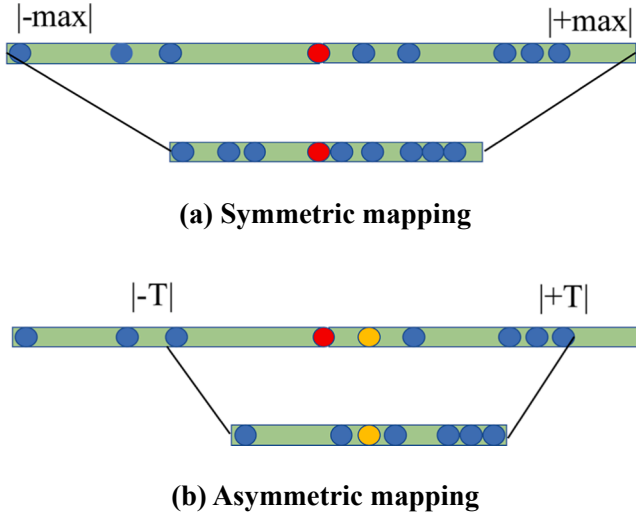


Fig. 6. Network parameter mapping method. (a):symmetric mapping. (b): asymmetric mapping. The red dot represents the midpoint, the yellow dot represents the adjusted midpoint, and the blue dot represents the parameter value. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the boundary of the region. as shown in Fig. 6(b). NVIDIA adjusts the threshold T of each layer through a calibration algorithm. The first step is to generate different quantitative distributions according to different thresholds. The second step is to calculate the relative entropy of each

distribution and the original distribution. The third step is to choose the one with the least entropy, which is the one most similar to the original distribution. Lower data precision can reduce memory occupation and improve efficiency, and get smaller model volume. This greatly reduces the model reasoning time.

2.8. Performance evaluation

The models are evaluated from 2 criteria, detection accuracy and operating speed. The mAP value and F1 value are used for model accuracy evaluation. In this research, we calculated the accuracy (AP) of strawberry, immature strawberry and flower separately. IoU represents the geometric relationship between the prediction box and the target box. The calculation of IoU is the ratio of the area of the intersection of the two boxes to the area of the union as Eq. (12) shows:

$$IoU = \frac{S(m_g \cap m_d)}{S(m_g \cup m_d)} \quad (12)$$

m_g is the ground truth bounding box, and m_d is the box obtained by the model's prediction on the picture. In the training process, by comparing the geometric relationship of the two boxes, the geometric relationship of the two boxes is converted into a loss function. Back-propagation is performed to further correct the geometric relationship between the two boxes, so that the two boxes are very close. Where S represents area, m_g represents true box, m_d represents prediction box. As shown in Fig. 7.

The IoU threshold is set to 0.5, and the samples are classified according to the threshold. Regarding the original sample, whether the original positive sample is predicted to be a positive sample is divided

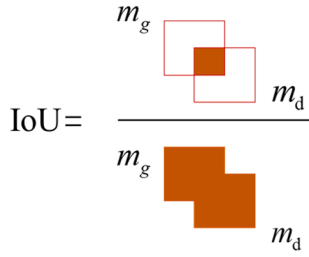


Fig. 7. Intersection over union representation.

into TP and FN. These two indicators define the precision P and the recall rate R, and the functions are as follows:

$$P = \frac{TP}{TP + FP} \quad (13)$$

$$R = \frac{TP}{TP + FN} \quad (14)$$

The precision P considers the proportion of positive samples that are correctly classified in all samples that are predicted to be positive. The recall rate R considers the proportion of correctly classified positive samples among all original positive samples.

Two indicators are considered separately and have certain limitations, so the points of precision and recall rate are combined to obtain the PR curve. The AP value is the area under the PR curve (Henderson and Ferrari, 2016), and the average accuracy mAP is obtained by averaging all types of AP values. In addition, the F1 value can also be used to assist in evaluating the accuracy of the model. The F1 value is calculated as Eq. (15).

$$F1 = \frac{2 * P * R}{P + R} \quad (15)$$

3. Results and discussion

3.1. Accuracy

The YOLO algorithm shows good performance on the COCO data set. The mAP and F1 values of YOLOv3 and YOLOv4 are higher than the YOLOv3-tiny and YOLOv4-tiny because YOLOv3 and YOLOv4 have more anchors to detect targets. We tested the mAP and F1 of 4 different YOLO models on the strawberry data set, as shown in Table 5. YOLOv3-tiny and YOLOv4-tiny are lightweight deep learning networks. The mAP of the YOLOv3-tiny model is significantly lower than the other 3 models. The mAP of YOLOv4-tiny is only 0.98% lower than YOLOv4. We obtain 4 new networks by optimizing the network structure of YOLOv4-Tiny. Network B reduces the CSPNet structure and the accuracy mAP decreases by 0.54%. Network C continues to reduce one convolution layer, and the network precision mAP decreases by 0.48%. The accuracy of networks B and C is almost equal.

On the basis of network C, CSPNet_F and CSPNet_S are used to replace the CSPNet to obtain networks D and E. The results are shown in Table 6. The mAP of D network reaches 82.38 %. The mAP of E network is 82.25 %, and the mAP of network C is 82.66 %. The mAP values of the 3 network structures are very close, and the difference is less than 0.5 %. In terms of F1 value, the biggest gap between the three is only 0.83 %.

Table 5
mAP and F1 of four YOLO series algorithms.

Network	mAP	F1
YOLOv3	83.26	81
YOLOv3-tiny	74.95	70.67
YOLOv4	83.88	81.5
YOLOv4-tiny	83.06	79.33

Table 6
mAP and F1 of five improved models.

Network	CBL	CSP	CSPNet_F	CSPNet_S	mAP	F1
A	3	3	\		83.06	79.33
B	4	2	\	\	82.61	77.67
C	3	2	\	\	82.66	79.33
D	3	\	2	\	82.38	78.67
E	3	\	\	2	82.25	79.00

The test result of model A, B, C, D and E are shown in the Figs. 9-13.

3.2. Efficiency

YOLOv3 and YOLOv4 have too many parameters, and they perform poorly in terms of efficiency. The speed of YOLOv3 is 34.45FPS on the PC, and only 2.0FPS on the Jetson Nano. The speed of YOLOv4 on PC is 25.06FPS, and it cannot be successfully loaded on Jetson Nano. This is because YOLOv4 have higher requirements for embedded devices, which increases difficulty to be deployed on embedded systems. YOLOv3-tiny and YOLOv4-tiny are lightweight networks with few network parameters, small model size, and superior performance in terms of speed. On the PC, the speed of YOLOv3-tiny is 134 FPS, and the speed of YOLOv4-tiny is 135FPS as shown in Table 7. In these four networks, whether on PC or Jetson Nano, the FPS of YOLOv3-tiny and YOLOv4-tiny is much larger than that of YOLOv3 and YOLOv4 models.

The PyTorch format model was first converted to TensorRT format model and then tested on Jetson Nano. The operating speed has been greatly improved. The speed of YOLOv4-tiny's PyTorch format model on Jetson Nano is only 11.0FPS, while the speed of TensorRT format model is 21.9FPS, as shown in Table 8. On PC, comparing with YOLOv4-tiny, the speed of Model B increases by 12FPS, and that of Model C increases by 25FPS. In model C, we modified the CSPNet of backbone network and obtain model D and E. For YOLOv4-tiny, the speed is increased by 31FPS and 35FPS, respectively. By simplifying the network structure and reducing the network parameters, the network speed increased accordingly. The TRT format models on Jetson Nano are taken as examples to demonstrate the relations of FLOPs and FPS of the 5 models as the Fig. 8 shows. On Jetson Nano, the speeds of model A, B, C, D and E in TRT formats are shown in the Figs. 9-13. We can see the speed of object detection is gradually increasing. The speed of model E is the highest, with PTH format of 13.1FPS and trt format of 25.2FPS.

By comparing the above two aspects, among the 4 improved models based on YOLOv4-tiny, model E has the highest speed in both formats

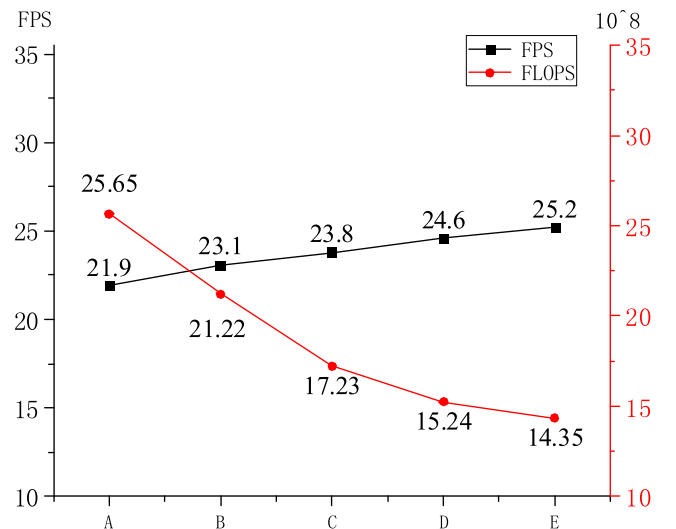


Fig. 8. Relationship between FLOPs and Speed FPS of Five Improved Network Models.



Fig. 9. YOLOv4-tiny: A (backbone with three CBL and three CSPNet).



Fig. 10. B (backbone with four CBL and two CSPNet).

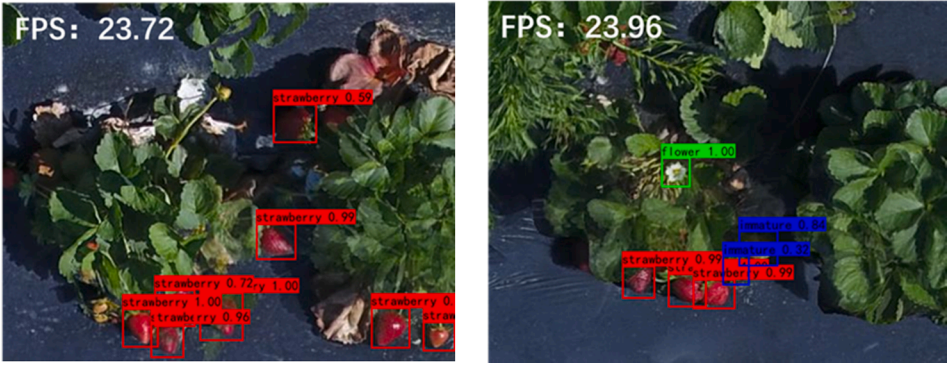


Fig. 11. C (backbone with three CBL and two CSPNet).

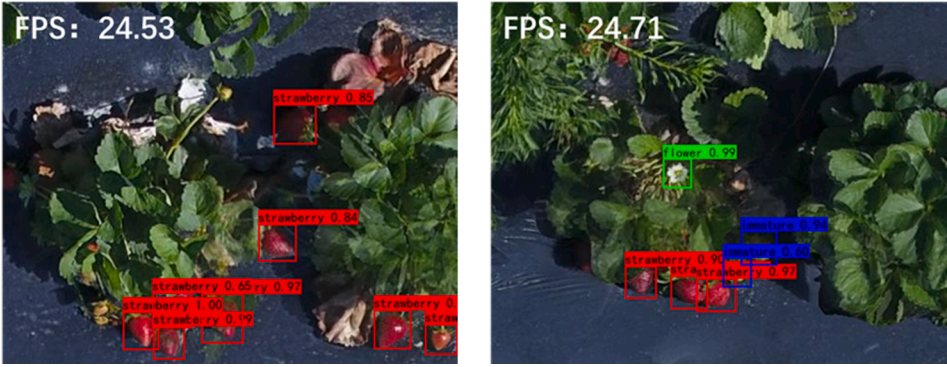


Fig. 12. D (backbone with three CBL and two CSPNet_F).



Fig. 13. RTSD-Net: E (backbone with three CBL and two CSPNet_S).

Table 7

Speed of four YOLO series algorithms.

Network	PTH (PC)	PTH (Jetson)	TRT (Jetson)
YOLOv3	34.45	2.0	4.43
YOLOv3-tiny	134	11.3	22.0
YOLOv4	25.06	Na	4.20
YOLOv4-tiny	135	11.0	21.9

and devices, and TRT Format has a speed of 25.2 FPS on Jetson Nano. In terms of accuracy, the difference is only 0.81 in mAP in contrast with YOLOv4-tiny. Therefore, Model E is finally selected as the real-time detection model of strawberry dataset, and Model E is named RTSD-Net. In terms of speed and accuracy, five network structures are compared in different formats and different devices. The results are shown Fig. 14.

3.3. Discussion

The accuracy of the YOLOv4-tiny model on the COCO dataset is only 40.2%, but due to its lightweight structure design, it can meet the requirements of real-time detection in terms of speed. Comparing with COCO dataset, the strawberry dataset used in this research has only three types of objects, and the image background is uniform. The recognition difficulty of the strawberry data set is greatly reduced than

COCO dataset. The mAP of YOLOv4-tiny for the strawberry data set is 83.06%. In the backbone extraction network of YOLOv4-tiny, we modified the network structure to reduce the complexity of the backbone network and the size of the whole network. After comprehensive experiments to evaluate the accuracy and speed of all the networks, the network with small precision reduction and obvious speed improvement was retained, and called RTSD-Net.

Training skills are important in building lightweight neural network. The strawberry detection model was trained on PC. Three training skills were used, namely cosine annealing learning rate, mosaic data augmentation and label smoothing. Mosaic data augmentation expands the data set, and the network learned more image features in one batch. Label smoothing prevents erroneous labels from interfering with model training. The cosine annealing learning rate fluctuates greatly, allowing the network to escape from the local optimal point. The three training techniques play an important role in model training.

Due to the limitations of cost and product volume, smart agricultural machinery is expected to use small, low power and inexpensive AI devices. Jetson Nano is an inexpensive embedded system, only 660 CNY (about 100 US dollar), and easy to get. Moreover, the deployment and compatibility of Jetson Nano is better than other devices like Google Coral which can make Jetson Nano an ideal platform for lightweight neural network developers.

Because capacity of the embedded device is much lower than that on PC, the speed of strawberry detection on the Jetson Nano was much

Table 8

Speed of five improved models.

Network	CBL	CSP	CSPNet_F	CSPNet_S	PTH (PC)	PTH (Jetson)	TRT (Jetson)
A	3	3	\	\	135	11	21.9
B	4	2	\	\	147	11.8	23.1
C	3	2	\	\	160	12.2	23.8
D	3	\	2	\	166	12.6	24.6
E	3	\	\	2	170	13.1	25.2

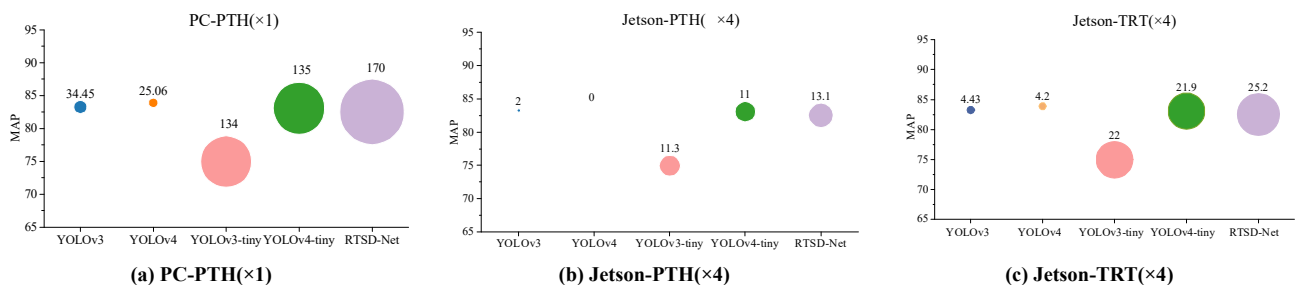


Fig. 14. The performance of mAP and FPS on different network. (a): the mAP and FPS of the network in PTH format on the PC. '×1' indicates that the graphics zoom ratio is one. (b): the mAP and FPS of the network in PTH format on the PC. (c): the mAP and FPS of the network in PTH format on the PC. '×4' indicates that the graphics zoom ratio is four.

lower than that on PC. The trt is a serialized format model, which is serialized by merging layers and reducing parameters. The recognition speed of the PTH format model of strawberry target detection in Jetson Nano was only 11FPS, which cannot meet the requirements of real-time object detection. After the transformation of the PTH format model trained on PC into the TensorRT model, the detection speed was doubled. Through the serialization of the model, the strawberry model can meet the requirements of both speed and accuracy on Jetson Nano, so as to achieve the real-time detection of strawberry.

4. Conclusion

Among the four existing YOLO networks, YOLOv4-tiny is the state-of-art deep neural network in object detection for edge computing as it is basically the same as YOLOv3 and YOLOv4 but it's much faster than the two model. So, it's selected as the basis of this research for further modifications to meet the demand of real-time strawberry detection. By gradually reducing the complexity of the network, the network parameters are reduced and the detection speed is improved. Among the 4 modified networks, it can be concluded that the computation amount of the model is negatively correlated to the speed of the model. The smaller the computation amount is, the faster the model is. Comparing with YOLOv4-tiny, the accuracy of RTSD-Net is only reduced by 0.62%. However, the detecting speed of RTSD-Net is greatly improved. While running on PC, the speed of RTSD-Net is increased by 35FPS, which is 25.93% higher than that of YOLOv4-tiny. The PyTorch format model is loaded on Jetson Nano and the speed of RTSD-Net is predicted to be 13.1FPS, which is 19.0% higher than that of YOLOv4-tiny. The model of RTSD-Net is converted to a TensorRT model, and is loaded on Jetson Nano. The speed of the TensorRT format model is 25.20fps, which is twice as fast as the PyTorch model, and 15% faster than the YOLOv4-tiny model. Combining the two aspects, the accuracy of RTSD-Net has only a slight decrease, but the speed has been significantly improved regardless of the type of device and the format of the model. RTSD-Net was proposed based on the lightweight neural network yolov4-tiny, and the speed is improved in the case of small accuracy decline. This has great advantages for edge computing, so as to realize real-time control of infield harvesting robot.

Funding

This research was funded by the National Natural Science Foundation of China (NSFC), grant number: 61905219.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Aguiar, A., Santos, F., Santos, L., Filipe, V., Sousa, A., 2020. Vineyard trunk detection using deep learning – An experimental device benchmark. *Comput. Electron. Agric.* 175.

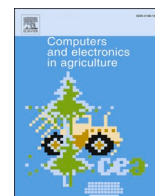
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*.
- Cass, S., 2020. Nvidia makes it easy to embed AI: The Jetson nano packs a lot of machine-learning power into DIY projects-[Hands on]. *IEEE Spectr.* 57 (7), 14–16.
- Chen, M., Tang, Y., Zou, X., Huang, Z., Zhou, H., Chen, S., 2021. 3D global mapping of large-scale unstructured orchard integrating eye-in-hand stereo vision and SLAM. *Comput. Electron. Agric.* 187, 106237. <https://doi.org/10.1016/j.compag.2021.106237>.
- Chen, Y., Lee, W.S., Gan, H., Peres, N., Fraisse, C., Zhang, Y., He, Y., 2019. Strawberry Yield Prediction Based on a Deep Neural Network Using High-Resolution Aerial Orthoimages. *Remote Sens.* 11 (13), 1584. <https://doi.org/10.3390/rs11131584>.
- DeVries, T., & Taylor, G.W., 2017. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*.
- Gao, J., French, A.P., Pound, M.P., He, Y., Pridmore, T.P., Pieters, J.G., 2020. Deep convolutional neural networks for image-based *Convolvulus sepium* detection in sugar beet fields. *Plant Methods* 16 (1). <https://doi.org/10.1186/s13007-020-00570-z>.
- He, K., Zhang, X., Ren, S., Sun, J., 2015. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (9), 1904–1916.
- Henderson, P., Ferrari, V., 2016. In: *End-to-end training of object class detectors for mean average precision*. Springer, pp. 198–213.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H., 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.
- Lin, G., Tang, Y., Zou, X., Cheng, J., Xiong, J., 2019. Fruit detection in natural environment using partial shape matching and probabilistic Hough transform. *Precis. Agric.*
- Liu, J., Wang, X., 2020. Early recognition of tomato gray leaf spot disease based on MobileNetV2-YOLOv3 model. *Plant Methods* 16 (1). <https://doi.org/10.1186/s13007-020-00624-2>.
- Mazzia, V., Khaliq, A., Salvetti, F., Chiaberge, M., 2020. Real-Time Apple Detection System Using Embedded Systems With Hardware Accelerators: An Edge AI Application. *IEEE Access* 8, 9102–9114.
- Müller, R., Kornblith, S., & Hinton, G., 2019. When does label smoothing help? *arXiv preprint arXiv:1906.02629*.
- Pound, M.P., Atkinson, J.A., Townsend, A.J., Wilson, M.H., Marcus, G., Jackson, A.S., Adrian, B., Georgios, T., Wells, D.M., Murchie, E.H., 2018. Deep Machine Learning provides state-of-the-art performance in image-based plant phenotyping. *Gigascience* 10, 1–10.
- Proteggente, A.R., Pannala, A.S., Paganga, G., Buren, L.V., Wagner, E., Wiseman, S., Put, F.V.d., Dacombe, C., Rice-Evans, C.A., 2002. The antioxidant activity of regularly consumed fruit and vegetables reflects their phenolic and vitamin C composition. *Free Radical Res. Commun.* 36 (2), 217–233.
- Redmon, J., & Farhadi, A., 2018. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
- Sati, V., Sánchez, S.M., Shoeibi, N., Arora, A., Corchado, J.M., 2020. In: *Face Detection and Recognition, Face Emotion Recognition Through NVIDIA Jetson Nano*. Springer, pp. 177–185.
- Vanholder, H., 2016. Efficient inference with tensorrt. In: ed.
- Vijitkunsawat, W., & Chantngarm, P., 2020. Comparison of machine learning algorithm's on self-driving car navigation using Nvidia Jetson Nano. In: 2020 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), IEEE, pp. 201–204.
- Wang, C.-Y., Liao, H.-Y.-M., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., Yeh, I.-H., 2020. CSPNet: A new backbone that can enhance learning capability of CNN. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp. 390–391.
- Wang, W., Xie, E., Song, X., Zang, Y., Wang, W., Lu, T., Yu, G., Shen, C., 2019. Efficient and accurate arbitrary-shaped text detection with pixel aggregation network. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8440–8449.
- Yun, S., Han, D., Oh, S.J., Chun, S., Choe, J., Yoo, Y., 2019. Cutmix: Regularization strategy to train strong classifiers with localizable features. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6023–6032.
- Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D., 2017. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412*.
- Distance-IOU loss: Faster and better learning for bounding box regression. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 12993–13000.

Update

Computers and Electronics in Agriculture

Volume 215, Issue , December 2023, Page

DOI: <https://doi.org/10.1016/j.compag.2023.108445>



Corrigendum to “Real-time strawberry detection using deep neural networks on embedded system (rtsd-net): An edge AI application” [Comput. Electron. Agric. 192 (2022) 106586]

Yanchao Zhang^{a,b,*}, Jiya Yu^a, Yang Chen^{b,c}, Won Suk Lee^d, Wen Yang^a, Wenbo Zhang^a, Yong He^{b,*}

^a School of Machinery Engineering and Automation, Zhejiang Sci-Tech University, Hangzhou, Zhejiang Province 310018, China

^b College of Biosystem Engineering and Food Science, Zhejiang University, Hangzhou, Zhejiang Province 310058, China

^c Zhejiang Lab, Hangzhou, Zhejiang Province 310018, China

^d Department of Agricultural and Biological Engineering, University of Florida, Gainesville, FL 32611, USA

The authors regret one author who has contributed significantly to this article was left out.

Correction in the author field has been updated as per author's instruction.

The download link at section 2.1 isn't available anymore and the source code will be shared by requests from readers.

The authors would like to apologise for any inconvenience caused.

DOI of original article: <https://doi.org/10.1016/j.compag.2021.106586>.

* Corresponding authors at: School of Machinery Engineering and Automation, Zhejiang Sci-Tech University, Hangzhou, Zhejiang Province 310018, China (Y. Zhang).

E-mail addresses: yczhang@zstu.edu.cn (Y. Zhang), yhe@zju.edu.cn (Y. He).

<https://doi.org/10.1016/j.compag.2023.108445>

Available online 25 November 2023

0168-1699/© 2023 Elsevier B.V. All rights reserved.